

Name: J. Mukhesh H.No: 2303A51813 Batch: 26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name		Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M. Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju	
Course Code	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week 3 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52
Duration	2 Hours	Applicable to Batches	All batches
Assignment Number: 8.3 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases Lab Objectives • Introduce TDD using AI • Generate test cases before implementation • Emphasize testing and validation • Encourage clean, reliable code Lab Outcomes Students will be able to: • Write AI-generated test cases	Week 4 - Wednesday	

- Implement code using test-first approach
- Validate using unittest
- Analyze test coverage
- Compare AI vs manual tests

Task 1: Email Validation using TDD

Scenario

You are developing a user registration system that requires reliable email input validation.

Requirements

- Must contain @ and . characters
- Must not start or end with special characters
- Should not allow multiple @ symbols
- AI should generate test cases covering valid and invalid email formats
- Implement is_valid_email(email) to pass all AI-generated test cases

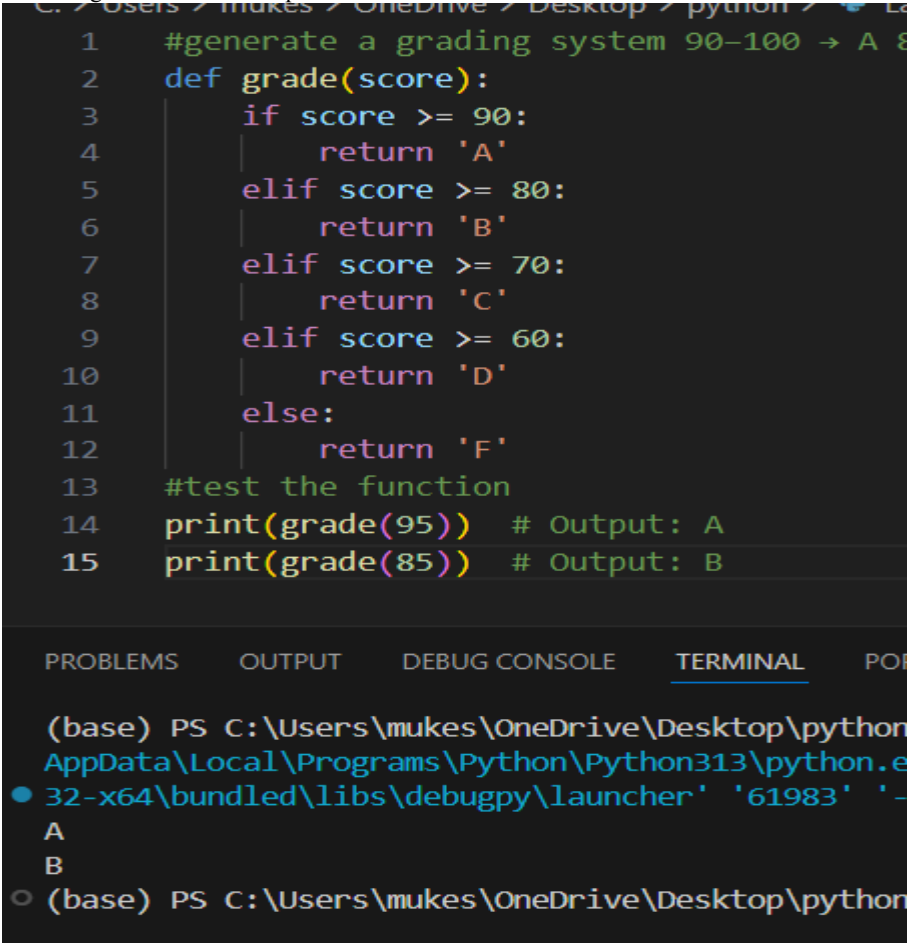
Expected Output

- Python function for email validation
- All AI-generated test cases pass successfully
- Invalid email formats are correctly rejected
- Valid email formats return True

```
C: > Users > mukes > OneDrive > Desktop > python > Lab10_3.py > ...
1  #generate a code the checks a email address is valid or not with
2  #1. It should contain exactly one "@" & "." symbol.
3  #2. Must start with a letter or number.
4  def is_valid_email(email):
5      # Check if there is exactly one "@" and one "." in the email
6      if email.count('@') != 1 or email.count('.') != 1:
7          return False
8
9      # Check if the email starts with a letter or number
10     if not email[0].isalnum():
11         return False
12
13     return True
14 # Test the function
15 email1 = "user@example.com"
16 print(is_valid_email(email1)) # Should return True
17
18 email2 = "user@@example.com"
19 print(is_valid_email(email2)) # Should return False
20
21 email3 = "user@example"
22 print(is_valid_email(email3)) # Should return False
23
24 email4 = "@example.com"
25 print(is_valid_email(email4)) # Should return False
26
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\mukes\OneDrive\Desktop\python> c::; cd 'c:\Users\mukes\OneDrive\Desktop\python'
AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\mukes\.vscode\
32-x64\bundled\libs\debugpy\launcher' '54201' '--' 'c:\Users\mukes\OneDrive\
True
False
False
False
(base) PS C:\Users\mukes\OneDrive\Desktop\python>
```

	Task 2: Grade Assignment using Loops Scenario You are building an automated grading system for an online examination platform. Requirements - AI should generate test cases for <code>assign_grade(score)</code> where: 90–100 → A 80–89 → B 70–79 → C 60–69 → D - Below 60 → F - Include boundary values (60, 70, 80, 90) - Include invalid inputs such as -5, 105, "eighty" - Implement the function using a test-driven approach Expected Output - Grade assignment function implemented in Python - Boundary values handled correctly - Invalid inputs handled gracefully - All AI-generated test cases pass  <pre> 1 #generate a grading system 90-100 → A & 2 def grade(score): 3 if score >= 90: 4 return 'A' 5 elif score >= 80: 6 return 'B' 7 elif score >= 70: 8 return 'C' 9 elif score >= 60: 10 return 'D' 11 else: 12 return 'F' 13 #test the function 14 print(grade(95)) # Output: A 15 print(grade(85)) # Output: B </pre> PROBLEMS OUTPUT DEBUG CONSOLE <u>TERMINAL</u> POW... <pre> (base) PS C:\Users\mukes\OneDrive\Desktop\python AppData\Local\Programs\Python\Python313\python.exe 32-x64\bundled\libs\debugpy\launcher' '61983' '- A B (base) PS C:\Users\mukes\OneDrive\Desktop\python </pre>	
	Task 3: Sentence Palindrome Checker Scenario You are developing a text-processing utility to analyze sentences. Requirements - AI should generate test cases for <code>is_sentence_palindrome(sentence)</code> - Ignore case, spaces, and punctuation - Test both palindromic and non-palindromic sentences	

	• Example: – "A man a plan a canal Panama" → True - Expected Output • Function correctly identifies sentence palindromes • Case and punctuation are ignored • Returns True or False accurately • All AI-generated test cases pass  <pre> 1 import re 2 def is_sentence_palindrome(sentence): 3 cleaned = re.sub(r'^a-zA-Z0-9+', '', sentence).lower() 4 return cleaned == cleaned[::-1] 5 test_cases = [6 ("A man a plan a canal Panama", True), 7 ("Racecar", True), 8 ("Was it a car or a cat I saw?", True), 9 ("Hello world", False), 10 ("This is not a palindrome", False), 11 ("", True), 12 ("a", True), 13 ("A", True), 14 ("ab", False), 15 ("aba", True), 16] 17 for sentence, expected in test_cases: 18 result = is_sentence_palindrome(sentence) 19 print(f'{sentence}' -> {result} (expected {expected})) 20 assert result == expected 21 print("All tests passed") </pre> <pre> PS C:\Users\shash> c:; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64904' '--' 'c:\U 'A man a plan a canal Panama' -> True (expected True) 'This is not a palindrome' -> False (expected False) '' -> True (expected True) 'a' -> True (expected True) 'A' -> True (expected True) 'ab' -> False (expected False) 'aba' -> True (expected True) All tests passed PS C:\Users\shash> </pre>	
	Task 4: ShoppingCart Class Scenario You are designing a basic shopping cart module for an e-commerce application. Requirements • AI should generate test cases for the ShoppingCart class	

Name:J.Mukhesh H.No:2303A51813 Batch:26

	• Class must include the following methods: – add_item(name, price) – remove_item(name) – total_cost() • Validate correct addition, removal, and cost calculation • Handle empty cart scenarios Expected Output • Fully implemented ShoppingCart class • All methods pass AI-generated test cases • Total cost is calculated accurately • Items are added and removed correctly	
--	---	--

Name:J.Mukhesh H.No:2303A51813 Batch:26

```
Welcome AAC A(8.3).py X
C: > Users > shash > AAC A(8.3).py > ShoppingCart > _init_
1 class ShoppingCart:
2     def __init__(self):
3         self.items = []
4     def add_item(self, name, price):
5         self.items.append((name, price))
6     def remove_item(self, name):
7         for i, (n, p) in enumerate(self.items):
8             if n == name:
9                 del self.items[i]
10                break
11    def total_cost(self):
12        return sum(price for name, price in self.items)
13    cart = ShoppingCart()
14    assert cart.total_cost() == 0
15    cart.add_item("apple", 1.0)
16    cart.add_item("banana", 2.0)
17    assert cart.total_cost() == 3.0
18    cart.add_item("apple", 1.0)
19    assert cart.total_cost() == 4.0
20    cart.remove_item("apple")
21    assert cart.total_cost() == 3.0
22    cart.remove_item("banana")
23    assert cart.total_cost() == 1.0
24    cart.remove_item("orange")
25    assert cart.total_cost() == 1.0
26    cart.remove_item("apple")
27    assert cart.total_cost() == 0
28    cart.add_item("milk", 3.5)
29    cart.add_item("bread", 2.5)
30    cart.add_item("milk", 3.5)
31    assert cart.total_cost() == 9.5
32    cart.remove_item("milk")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

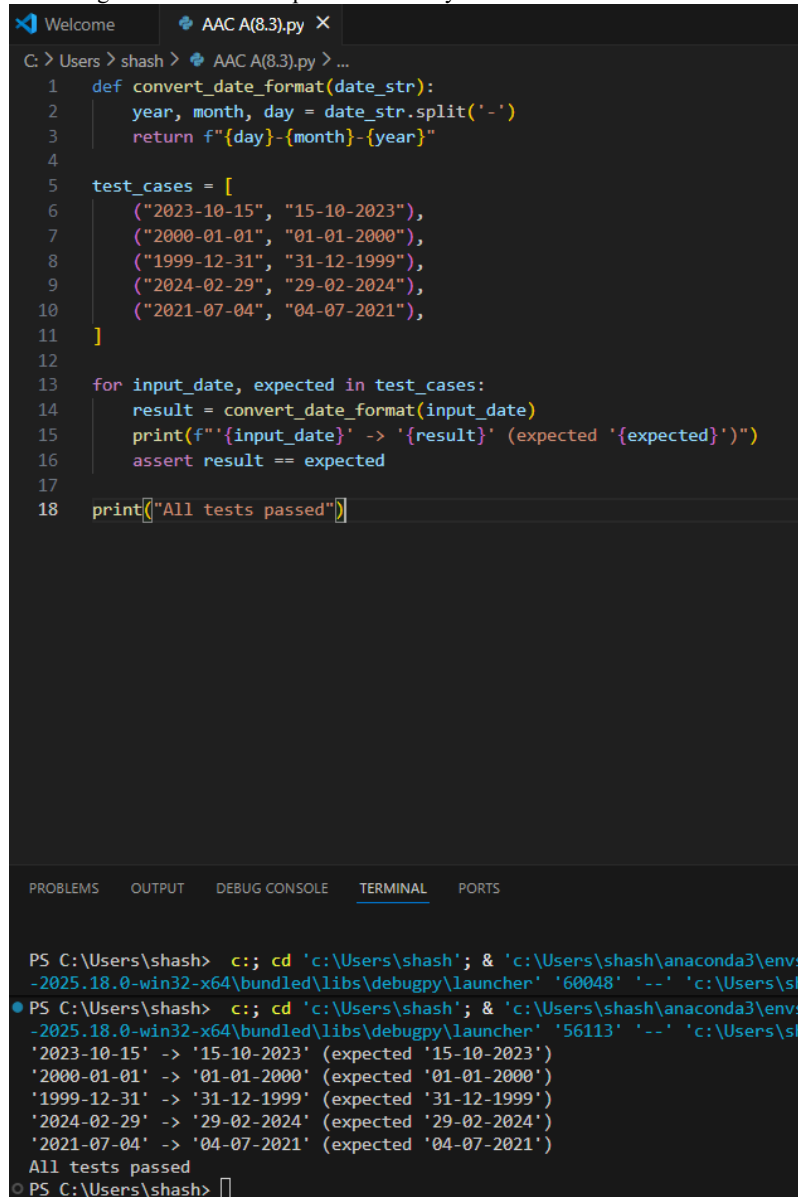
```
PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\an
-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64904' '--'
'A man a plan a canal Panama' -> True (expected True)
'A' -> True (expected True)
'ab' -> False (expected False)
'aba' -> True (expected True)
All tests passed
PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\an
-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60048' '--'
All tests passed
PS C:\Users\shash>
```

	 <pre> Welcome AAC A(8.3).py X C: > Users > shash > AAC A(8.3).py > ShoppingCart > _init_ 1 class ShoppingCart: 6 def remove_item(self, name): 10 break 11 def total_cost(self): 12 return sum(price for name, price in self.items) 13 cart = ShoppingCart() 14 assert cart.total_cost() == 0 15 cart.add_item("apple", 1.0) 16 cart.add_item("banana", 2.0) 17 assert cart.total_cost() == 3.0 18 cart.add_item("apple", 1.0) 19 assert cart.total_cost() == 4.0 20 cart.remove_item("apple") 21 assert cart.total_cost() == 3.0 22 cart.remove_item("banana") 23 assert cart.total_cost() == 1.0 24 cart.remove_item("orange") 25 assert cart.total_cost() == 1.0 26 cart.remove_item("apple") 27 assert cart.total_cost() == 0 28 cart.add_item("milk", 3.5) 29 cart.add_item("bread", 2.5) 30 cart.add_item("milk", 3.5) 31 assert cart.total_cost() == 9.5 32 cart.remove_item("milk") 33 assert cart.total_cost() == 6.0 34 print("All tests passed") PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS PS C:\Users\shash> c:; cd 'c:\Users\shash'; & 'c:\Users\shash -2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64904' '- 'A man a plan a canal Panama' -> True (expected True) 'A' -> True (expected True) 'ab' -> False (expected False) 'aba' -> True (expected True) All tests passed PS C:\Users\shash> c:; cd 'c:\Users\shash'; & 'c:\Users\shash -2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60048' '- All tests passed PS C:\Users\shash> </pre>	
	Task 5: Date Format Conversion Scenario You are creating a utility function to convert date formats for reports. Requirements - AI should generate test cases for <code>convert_date_format(date_str)</code> - Input format must be "YYYY-MM-DD" - Output format must be "DD-MM-YYYY" - Example:	

- "2023-10-15" → "15-10-2023"

Expected Output

- Date conversion function implemented in Python
- Correct format conversion for all valid inputs
- All AI-generated test cases pass successfully



```
1 def convert_date_format(date_str):
2     year, month, day = date_str.split('-')
3     return f"{day}-{month}-{year}"
4
5 test_cases = [
6     ("2023-10-15", "15-10-2023"),
7     ("2000-01-01", "01-01-2000"),
8     ("1999-12-31", "31-12-1999"),
9     ("2024-02-29", "29-02-2024"),
10    ("2021-07-04", "04-07-2021"),
11]
12
13 for input_date, expected in test_cases:
14     result = convert_date_format(input_date)
15     print(f'{input_date} -> {result} (expected {expected})')
16     assert result == expected
17
18 print("All tests passed")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\
-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '60048' '--' 'c:\Users\sh
● PS C:\Users\shash> c::; cd 'c:\Users\shash'; & 'c:\Users\shash\anaconda3\envs\
-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56113' '--' 'c:\Users\sh
'2023-10-15' -> '15-10-2023' (expected '15-10-2023')
'2000-01-01' -> '01-01-2000' (expected '01-01-2000')
'1999-12-31' -> '31-12-1999' (expected '31-12-1999')
'2024-02-29' -> '29-02-2024' (expected '29-02-2024')
'2021-07-04' -> '04-07-2021' (expected '04-07-2021')
All tests passed
PS C:\Users\shash> █
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.